objects are to be preferred (for instance, an object with a `min` and `max` field would be clearer in the example above).

Tuples, however, provide a viable and synthetic alternative whenever the meaning of the fields is clear enough. To exploit this even further, we use a particular notation to assign all or part of the fields of an object to a tuple. Assume we have an object `Employee` with fields `name`, `surname`, `age`, `address` etc...

If `empl` is an instance of employee, we use the following syntax to extract any subset of fields from `empl` into *aliases*[3]:

```
(name, surname, age) ← empl
```

Of course, in the example above, we extract only three fields.

This syntax is particularly convenient in combination with `for-each` loops, as we can iterate through a collection of employees and access directly aliases to the fields we need, without having to redundantly write something like `empl.name` each time we access a field of the `empl` object. Here it is a comparative example to make the difference clear:

```
for empl in employees do
  user ← empl.surname + empl.age

for (surname, age) in employees do
```